

CardioSurv

Remaining Task Assignments — tasks_2

Cardiovascular Risk Classification, Treatment Recommendation, Survival Prediction

Course: AIT201 Applied Machine Learning, Xiamen University Malaysia

Deadline: 25 June 2026 | Focus: Coding only (report & slides excluded)

Task	Owner	Status	Priority
T-A: Part 2 Recommender + Survival Model	BOUGACHA MOHAMED	NOT STARTED	HIGH
T-B: Evaluation Figures (metrics.py + plots.py)	TAN GUAN HAN	NOT STARTED	HIGH
T-C: Frontend (HTML/CSS/JS)	LAW ZI YING	NOT STARTED	HIGH
T-D: Production Deploy + Error Handling	CHILUBA CHIFUNILO	MEDIUM	MEDIUM
T-E: Part 2 API Wiring (real /recommend)	CHILUBA CHIFUNILO	NOT STARTED	MEDIUM

Already completed (do not redo):

- Task 1 — Ang Jing Ru: engineering.py + features.csv ■
- Task 2 — Bougacha: part1_classifier.py + .pkl + notebook ■
- Task 3 — Law Zi Ying: FastAPI scaffold (already upgraded to real wiring) ■
- Task 4 — Chiluba: DB models + session + seed + real API wiring ■
- Task 5 — Tan Guan Han: EDA notebook + 5 figures + test_bed.py + test_grace.py ■

Read first: Every task references docs/schemas.md and docs/api_contract.md. These are the locked Day-1 contracts.

Execution Order — Who Starts First

The tasks below are ordered by dependency. Start at Phase 1 and move down. Tasks in the same phase can run in parallel.

Phase 1 — Start immediately (no dependencies)

- BOUGACHA — T-A: Part 2 Recommender + Survival Model
- TAN GUAN HAN — T-B: metrics.py + plots.py (evaluation scaffold)
- LAW ZI YING — T-C: Frontend HTML skeleton + api.js

Phase 2 — After T-A is done

- CHILUBA — T-E: Wire real /api/v1/recommend (needs T-A .pkl)

Phase 3 — After T-E + T-C are done

- CHILUBA — T-D: Production deploy to Render + error handling
- LAW ZI YING — T-C cont.: Switch frontend to real prod URL

Phase 4 — Whole-team integration test

- Run 4 example patients end-to-end through the live API
- Verify DB rows, real predictions, real recommendations
- Frontend shows correct results on mobile + desktop

A BOUGACHA MOHAMED — Part 2 Recommender + Survival Model HIGH

PRIORITY

[NOT STARTED — Start immediately]

Role. The second ML model in the pipeline. Takes the Part 1 risk category + routing output and predicts intervention type, intensity, and survival probability. Chiluba cannot wire `/api/v1/recommend` with real predictions until this is done.

What to build

Since MIMIC-IV access is not guaranteed, build the Part 2 model using the existing `features.csv` augmented with the routing output from `src/clinical/routing.py`. Train an XGBoost classifier for intervention type recommendation. Build a survival model using **lifelines** (Kaplan-Meier + Cox PH). Expose a clean `recommend(features_dict)` function the API can call.

Files to create or modify

Action	File / Path
CREATE	<code>src/models/part2_recommender.py</code> (currently empty – 0 bytes)
CREATE	<code>notebooks/03_part2_modeling.ipynb</code>
CREATE	<code>models/part2_recommender_v1.0.pkl</code>
CREATE	<code>models/survival_cox_v1.0.pkl</code>

Functions to implement

- `build_part2_features(df, route_results)` — Merge `features.csv` columns with routing output (intervention_type, grace_score, bed_gy, medication_intensity). Returns a DataFrame ready for XGBoost training.
- `train_xgboost_recommender(X, y)` — XGBoost classifier predicting intervention_label (0=Medication-Standard, 1=Medication-Intensified, 2=Medication-Maximal, 3=SBRT). `n_estimators=300`, `max_depth=5`, `random_state=42`.
- `train_survival_model(df)` — Use lifelines. Fit KaplanMeierFitter stratified by intervention branch. Fit CoxPHFitter with age, risk_category_encoded, grace_score, bed_gy as covariates. Returns (kmf, cph) tuple.
- `predict_survival(patient_dict, cph)` — Returns survival_without (no treatment, baseline) and survival_with (recommended treatment) as 2-year probabilities (0.0-1.0).
- `recommend(features_dict)` — Main API function. Calls `routing.route_patient()` internally, then XGBoost for intervention, then survival model. Returns the JSON shape from `schemas.md §5`.
- `save(bundle, path) / load(path)` — joblib wrappers. Bundle must include: `xgb_pipeline`, `kmf`, `cph`, `label_encoder`, `version`.
- `main()` — Load `features.csv` → build Part 2 features → 80/20 split → train both models → evaluate → save bundles → smoke-test `recommend()`.

recommend() output shape (from schemas.md §5)

```
{
  "patient_id":      "",
  "intervention_type": "SBRT",           // or "Medication"
  "intensity_level": "Maximal",          // Standard / Intensified / Maximal / N/A
  "survival_without": 0.61,              // 2-year survival, no treatment
```

```

"survival_with":    0.84,                // 2-year survival, with treatment
"grace_score":      142,
"bed_gy":           87.5,                // null if Medication path
"routing_path":     "High-Risk Path: Cardiac Radioablation (SBRT)",
"model_version":    "part2_recommender_v1.0"
}

```

Snippet to start from

```

from src.clinical.routing import route_patient, RiskLevel
from src.models.part1_classifier import load as load_part1, predict as part1_predict
import xgboost as xgb
from lifelines import KaplanMeierFitter, CoxPHFitter

def recommend(features_dict: dict, bundle=None) -> dict:
    if bundle is None:
        bundle = load("models/part2_recommender_v1.0.pkl")
    # 1. Get Part 1 risk
    p1 = part1_predict(features_dict)
    # 2. Route patient
    route = route_patient(features_dict, p1["risk_category"], p1["confidence"])
    # 3. XGBoost intervention recommendation
    # 4. Survival prediction
    # 5. Return full JSON
    ...

```

Expected output

```

$ python -m src.models.part2_recommender
[part2] Loading features.csv ...
[part2] Train: 1136 rows, Test: 285 rows
[part2] XGBoost recommender → acc=0.82 f1_macro=0.79
[part2] Cox PH model → concordance=0.71
[part2] Saved models/part2_recommender_v1.0.pkl
[part2] Saved models/survival_cox_v1.0.pkl
[part2] Smoke-test: High-risk male 67 → SBRT, survival 0.61 → 0.84

```

Any XGBoost accuracy above 0.75 is acceptable. Cox concordance index above 0.65 is acceptable.

Depends on	Nothing. features.csv and routing.py are already on GitHub. Start immediately.
Blocks	Chiluba T-E (real /api/v1/recommend wiring).

B TAN GUAN HAN — Evaluation Figures (metrics.py + plots.py) HIGH PRIORITY

[NOT STARTED — Start immediately]

Role. Quality assurance and visual output. Both files are currently 0 bytes. These functions are called by Bougacha's notebook and by the report team to generate print-resolution figures.

What to build (two parts)

Part A — src/evaluation/metrics.py. Two functions that compute and return evaluation metrics as dicts. Used by both Part 1 and Part 2 model notebooks.

Part B — src/evaluation/plots.py. Four plotting functions that save publication-quality PNGs to reports/figures/. Must use a consistent colour palette (define hex values at the top of the file).

Files to create or modify

Action	File / Path
MODIFY	src/evaluation/metrics.py (currently empty – 0 bytes)
MODIFY	src/evaluation/plots.py (currently empty – 0 bytes)
CREATE	reports/figures/confusion_matrix.png
CREATE	reports/figures/roc_curve.png
CREATE	reports/figures/km_curves.png
CREATE	reports/figures/feature_importance.png

Functions to implement — metrics.py

- `classification_metrics(y_true, y_pred, y_proba, labels)` — Returns dict with keys: accuracy, f1_macro, auc_ovr, per_class_f1 (dict), confusion_matrix (nested list). labels = ['Low','Medium','High'].
- `survival_metrics(durations, events, predicted_scores)` — Returns dict with keys: concordance_index (C-index from lifelines), log_rank_p (from lifelines logrank_test). Accepts arrays.

Functions to implement — plots.py

- `plot_confusion_matrix(cm, labels, save_path)` — Seaborn heatmap, annotated, colour palette from the defined palette. save_path default: reports/figures/confusion_matrix.png. Returns fig.
- `plot_roc_curve(y_true, y_proba, labels, save_path)` — One ROC curve per class (OvR), micro-average, macro-average. Legend shows AUC per class. save_path default: reports/figures/roc_curve.png.
- `plot_km_curves(kmf_dict, save_path)` — Kaplan-Meier curves stratified by treatment branch. kmf_dict = {'SBRT': kmf1, 'Medication': kmf2}. Include 95% confidence bands. save_path default: reports/figures/km_curves.png.
- `plot_feature_importance(importance_df, save_path)` — Horizontal bar chart. importance_df has columns ['feature','importance']. Top 15 features. save_path default: reports/figures/feature_importance.png.

Snippet to start from

```
# src/evaluation/metrics.py
from sklearn.metrics import accuracy_score, f1_score, roc_auc_score, confusion_matrix
from lifelines.statistics import logrank_test
from lifelines.utils import concordance_index

PALETTE = {
    "Low": "#2ecc71",
    "Medium": "#f39c12",
    "High": "#e74c3c",
    "primary": "#3498db",
    "secondary": "#9b59b6",
}

def classification_metrics(y_true, y_pred, y_proba, labels) -> dict:
    return {
        "accuracy": round(accuracy_score(y_true, y_pred), 4),
        "f1_macro": round(f1_score(y_true, y_pred, average="macro"), 4),
        "auc_ovr": round(roc_auc_score(y_true, y_proba, multi_class="ovr"), 4),
        "confusion_matrix": confusion_matrix(y_true, y_pred).tolist(),
    }
```

Expected output

```
$ python -c "from src.evaluation.metrics import classification_metrics; print('ok')"
ok

$ python -c "from src.evaluation.plots import plot_confusion_matrix; print('ok')"
ok
```

Run each function on mock data to verify the PNG files are created without errors.

Depends on	Nothing. Can start immediately using mock arrays. Wire to real model outputs after T-A is done.
Blocks	Bougacha's 03_part2_modeling.ipynb (needs plot functions). Report team figures.

C LAW ZI YING — Frontend Interface (HTML / CSS / JS) HIGH PRIORITY

[NOT STARTED — Start immediately]

Role. The user-facing layer. No frontend folder exists in the repo at all — this is a complete greenfield build. The backend API is already live locally so you can hit real endpoints from Day 1.

What to build

Part A — Structure & Skeleton (start here)

- Create frontend/ folder at repo root
- frontend/index.html — landing page with project intro and a 'Start Assessment' button
- frontend/predict.html — patient input form with all 11 fields from the API contract
- frontend/results.html — results display page
- frontend/history.html — prediction history table
- frontend/css/styles.css — CSS variable system for colours and spacing
- frontend/js/api.js — fetch() wrappers for all 5 API endpoints

Files to create

Action	File / Path
CREATE	frontend/index.html
CREATE	frontend/predict.html
CREATE	frontend/results.html
CREATE	frontend/history.html
CREATE	frontend/css/styles.css
CREATE	frontend/js/api.js

predict.html — required input fields

Field	Type	UI Element
age	int	number input, min=1 max=120
sex	str	dropdown: M / F
chest_pain_type	str	dropdown: TA / ATA / NAP / ASY
resting_bp	int	number input, min=50 max=250
cholesterol	int	number input, min=50 max=800
fasting_bs	int	radio: 0 / 1
resting_ecg	str	dropdown: Normal / ST / LVH
max_hr	int	number input, min=40 max=230
exercise_angina	str	dropdown: N / Y
oldpeak	float	number input, step=0.1, min=-3 max=7
st_slope	str	dropdown: Up / Flat / Down
has_arrhythmia	bool	checkbox (for /recommend)

results.html — required display elements

- Risk category badge — colour coded (green=Low, orange=Medium, red=High) AND shape coded (circle=Low, triangle=Medium, square=High) for accessibility
- Confidence progress bar (e.g. 83% filled)
- Treatment recommendation card (intervention_type, intensity_level)
- Survival comparison panel — two numbers: survival_without vs survival_with as percentages
- Chart.js bar chart comparing survival_without vs survival_with (import from CDN)

history.html — required display elements

- Paginated table: date, patient_id, risk_category, intervention_type, survival improvement
- Clicking any row opens a modal that fetches GET /api/v1/patients/{patient_id} and shows full record
- Empty state message when no history exists

api.js — functions to implement

```
const API_BASE = "http://localhost:8000"; // one-line change for prod

async function getHealth()           { ... } // GET  /api/v1/health
async function submitPredict(body)   { ... } // POST /api/v1/predict
async function submitRecommend(body){ ... } // POST /api/v1/recommend
async function getHistory(page=1)    { ... } // GET  /api/v1/history
async function getPatient(id)        { ... } // GET  /api/v1/patients/{id}

// All functions return parsed JSON. On error, throw with status code.
```

Client-side validation required (before any fetch call)

- All fields present and non-empty
- age between 1–120, resting_bp between 50–250, etc. (match API Pydantic constraints)
- Show inline error messages next to each invalid field — do not just alert()
- Disable the Submit button while a fetch is in flight (show loading state)

CORS note

The API already has CORS configured for *. Use Live Server (VS Code) or `python -m http.server 5500` from the frontend/ folder to serve locally.

Depends on	Nothing. The mock API is already live at <code>http://localhost:8000</code> . Start immediately against it.
Blocks	Chiluba T-D (needs frontend URL to whitelist in CORS for prod deploy).

D CHILUBA CHIFUNILO — Production Deploy + Error Handling MEDIUM

PRIORITY

[Do Part A (error handling) now. Part B (deploy) after T-E is done.]

Role. Makes the API production-ready and publicly accessible. T-E (real /recommend wiring) should be done first so the prod deploy includes the full system.

Part A — Error handling + rate limiting (do first, no dependencies)

Modify `src/api/main.py` — these are gaps in the current implementation:

Action	File / Path
MODIFY	<code>src/api/main.py</code> — add structured error responses + rate limiting
CREATE	<code>docs/deployment.md</code> — deploy runbook
MODIFY	<code>.env.example</code> — add RENDER env vars

What to add to `main.py`

- Rate limiting on POST `/api/v1/predict` using `slowapi` (already in requirements). Limit: 30 requests/minute per IP. Return 429 with a clear message if exceeded.
- Structured 422 validation error responses — `Pydantic` already handles this, but add a custom exception handler so the body shows which field failed and why.
- Model loaded ONCE at application startup using FastAPI lifespan event (not on every request). The current `main.py` may load the `.pkl` on each call — fix this.
- Global 500 handler that returns `{"error": "Internal server error", "detail": str(exc)}` instead of a raw traceback.

Snippet — startup model loading

```
from contextlib import asynccontextmanager

MODEL_BUNDLE = {}

@asynccontextmanager
async def lifespan(app: FastAPI):
    # Load once at startup
    MODEL_BUNDLE["part1"] = load_part1("models/part1_classifier_v1.0.pkl")
    MODEL_BUNDLE["part2"] = load_part2("models/part2_recommender_v1.0.pkl")
    yield
    MODEL_BUNDLE.clear()

app = FastAPI(title="CardioSurv API", lifespan=lifespan)
```

Part B — Deploy to Render (do after T-E is merged)

- Create Render Web Service from GitHub repo — Python 3.11, build: `pip install -r requirements.txt`, start: `uvicorn src.api.main:app --host 0.0.0.0 --port $PORT`
- Provision Render PostgreSQL free tier. Set `DATABASE_URL` env var.
- Set `MODEL_DIR`, `CORS_ORIGINS` (include frontend Render Static Site URL) env vars.
- Enable auto-deploy on push to main.
- Run DB init via Render shell: `python -c "from src.db.session import engine; from src.db.models import Base; Base.metadata.create_all(engine)"`

- Smoke-test the live URL with curl against all 5 endpoints.
- Write docs/deployment.md: deploy steps, how to check logs, how to keep prod warm (ping /health every 10 min before demo — Render free tier sleeps after 15 min idle).

Expected output

```
$ curl https://cardiosurv-api.onrender.com/api/v1/health
{"status":"ok","version":"v1","model_versions":{"part1_classifier":"v1.0","part2_recommender":"v1.0"},...}

$ curl -X POST https://cardiosurv-api.onrender.com/api/v1/predict \
-H "Content-Type: application/json" \
-d '{"age":56,"sex":"M","chest_pain_type":"ATA",...}'
{"patient_id":"","risk_category":"Medium","confidence":0.83,...}
```

Depends on	Part A: nothing. Part B: needs T-E (real /recommend) to be merged first.
Blocks	No one. You are the final integration point.

E CHILUBA CHIFUNILO — Wire Real /api/v1/recommend MEDIUM PRIORITY

[Do after Bougacha's T-A is merged (part2_recommender.pkl exists)]

Role. Replace the current routing.py mock in /api/v1/recommend with a real call to Bougacha's part2_recommender.recommend(). This is the final integration milestone before production deploy.

What to build

The current src/api/main.py already has a /api/v1/recommend endpoint. It currently calls src/clinical/routing.py directly and returns a partial response. Replace its body with a real flow:

Action	File / Path
MODIFY	src/api/main.py – replace mock recommend logic with real part2 call

Real flow for POST /api/v1/recommend

- Step 1: Validate input with Pydantic RecommendRequest
- Step 2: Look up existing patient row in DB by patient_id (if provided) OR insert new row
- Step 3: Call part2_recommender.recommend(features_dict) → get full recommendation dict
- Step 4: Insert a recommendations row in DB with intervention_type, intensity_level, survival_without, survival_with
- Step 5: Return RecommendResponse (full JSON shape from api_contract.md §3)

Snippet to start from

```
from src.models.part2_recommender import recommend as part2_recommend

@app.post("/api/v1/recommend", response_model=RecommendResponse)
def recommend_endpoint(
    body: RecommendRequest,
    db: Session = Depends(get_db)
):
    features = body.model_dump()
    result = part2_recommend(features, bundle=MODEL_BUNDLE.get("part2"))
    # insert recommendations row
    rec = Recommendation(
        patient_id=result["patient_id"],
        intervention_type=result["intervention_type"],
        intensity_level=result["intensity_level"],
        survival_without=result["survival_without"],
        survival_with=result["survival_with"],
    )
    db.add(rec); db.commit()
    return result
```

Expected output

```
$ curl -X POST http://localhost:8000/api/v1/recommend \
-H "Content-Type: application/json" \
-d '{"age":67,"sex":"M","chest_pain_type":"ASY","resting_bp":162,
    "cholesterol":268,"fasting_bs":1,"resting_ecg":"ST","max_hr":100,
    "exercise_angina":"Y","oldpeak":2.5,"st_slope":"Flat",
```

```
        "has_arrhythmia":true}'  
  
{  
  "patient_id": "",  
  "intervention_type": "SBRT",  
  "intensity_level": "N/A",  
  "survival_without": 0.61,  
  "survival_with": 0.84,  
  "grace_score": null,  
  "bed_gy": 87.5,  
  "routing_path": "High-Risk Path: Cardiac Radioablation (SBRT)",  
  "model_version": "part2_recommender_v1.0"  
}
```

A row must appear in the recommendations table in cardiosurv.db.

Depends on	Bougacha T-A: needs part2_recommender_v1.0.pkl and the recommend() function.
Blocks	Chiluba T-D Part B (production deploy).

Phase 4 — Whole-Team Integration Test ALL MEMBERS

Run this after T-A, T-B, T-C, T-D, and T-E are all merged. Every member participates. This is the final sign-off before submission.

4 Example Cases — run all through the live API

Case 1 — High risk, arrhythmia

```
{ "age": 67, "sex": "M", "chest_pain_type": "ASY", "resting_bp": 162, "cholesterol": 268, "fasting_bs": 1, "resting_ecg": "ST", "max_hr": 100, "exercise_angina": "Y", "oldpeak": 2.5, "st_slope": "Flat", "has_arrhythmia": true }
```

Expected: Expected: High risk → SBRT, survival improvement visible

Case 2 — Medium risk, no arrhythmia

```
{ "age": 55, "sex": "M", "chest_pain_type": "NAP", "resting_bp": 138, "cholesterol": 230, "fasting_bs": 0, "resting_ecg": "Normal", "max_hr": 150, "exercise_angina": "N", "oldpeak": 1.2, "st_slope": "Up", "has_arrhythmia": false }
```

Expected: Expected: Medium risk → Medication-Intensified

Case 3 — Low risk female

```
{ "age": 44, "sex": "F", "chest_pain_type": "TA", "resting_bp": 120, "cholesterol": 195, "fasting_bs": 0, "resting_ecg": "Normal", "max_hr": 170, "exercise_angina": "N", "oldpeak": 0.0, "st_slope": "Up", "has_arrhythmia": false }
```

Expected: Expected: Low risk → Medication-Standard

Case 4 — Elderly, multiple risk factors

```
{ "age": 76, "sex": "M", "chest_pain_type": "ASY", "resting_bp": 180, "cholesterol": 310, "fasting_bs": 1, "resting_ecg": "ST", "max_hr": 88, "exercise_angina": "Y", "oldpeak": 3.5, "st_slope": "Flat", "has_arrhythmia": true }
```

Expected: Expected: High risk → SBRT, high BED, large survival delta

Integration checklist

- POST /api/v1/predict returns real risk_category and confidence for all 4 cases
- POST /api/v1/recommend returns real intervention_type and survival values for all 4 cases
- Each predict call writes a row to the patients table and predictions table
- Each recommend call writes a row to the recommendations table
- GET /api/v1/history returns the records just inserted
- GET /api/v1/patients/{id} returns full record for a patient just created
- Frontend predict form submits and navigates to results page correctly
- Frontend results page shows correct badge colour and survival chart
- Frontend history page shows the 4 test records
- Production URL (Render) responds to all 5 endpoints
- pytest tests/ -v passes with 0 failures

End of tasks_2. Questions: bring to the next team sync.